

Formal Methods for Software Engineering homework report

Authors: Davide Greco, Edoardo Egidio

Table of Contents

1. Design choices	2
1.1. Master-slave approach	2
1.2. World movement (symbolic)	2
1.3. Player modeling (symbolic)	3
2. Symbolic version results	4
2.1. First query	4
2.2. Second query	5
2.2.1. Assignment	5
2.3. Third query	5
2.4. Fourth query	5
2.5. Fifth query	6
2.6. Sixth query	6
2.7. Seventh query	6
3. Stochastic version	6
3.1. Queries	8
3.1.1. First query	9
3.1.2. Second query	9
Bibliography	12

1. Design choices

1.1. Master-slave approach

Since at every tick both the world and the player must move, we decided to take a master-slave approach: at each tick, the world moves first, and notifies a global broadcast channel, that the player listens to. As soon as the world ticked, the player performs its “turn”, which is:

- checking if the world movement killed him
- if not, perform a movement (which can either be to go up or to stay still)
- check if the movement killed him:
 - if so, we must reset
 - if not, we go back to waiting state (unless we are in `MAX_Y` row, in which case we just occupied a home bay and we must reset)

This way we are sure that the world and player movement are synchronized, and that there is never a configuration that would kill the player that goes unnoticed.

1.2. World movement (symbolic)

Since all elements of a row move at the same speed (at least for the symbolic version, see [Section 3](#)), to reduce state-space size we decided to model the world as an array of rows that move with the speed of the elements occupying the given row. To achieve this we used:

- A matrix containing the initial state of the world: each row has 18 + 2 columns, to handle the warping that takes two more steps. Each cell is an int representing the type of the cell, which can be

Number	Name	Description
−1	<code>C_WARP</code>	not a real cell, but the space of warping
0	<code>C_EMPTY</code>	a cell in which there is no entity (not considering the player)
1	<code>C_OBJ</code>	either a car (in the first 6 rows), or a wall (in the last row)
2	<code>C_LOG</code>	a log (only in rows 7-11)
3	<code>C_TURTLE</code>	a regular (i.e. not diving) turtle (only in rows 7-11)
4	<code>C_DIVING</code>	a diving turtle (only in rows 7-11)

- An array of speeds (one for each row). As per specification, the speed of a row is the number of ticks between two consecutive movements.
- An array of directions (one for each row). When we want to get a specific cell, we need to multiply the current offset with the direction of the current row. We must however pay attention to a detail: without loss of generality, let’s assume that we

are looking at a row that moves from left to right. Since we are moving the whole row, when it moves all its elements will be shifted to the right by one, so the element now occupying cell x is the element in the original matrix occupying cell $x-1$: this means that we must use -1 for left-to-right movement, and 1 for right-to-left movement.

- An array of offsets (one for each row). Each tick, if the current number of ticks is divisible by the speed of the current row, this will increase by one (with modulo `MATRIX_COLS`).
- The function

```
int[-1,4] get_cell(int[0, MAX_Y] row, int[0, MATRIX_COLS] col)
```

which makes use of all the aforementioned structures to get the type of cell currently occupying a given position.

In this way, we have a single automata for all the rows that, at each ticks, updates all the rows (and hence all the elements of the rows). Moreover, it also results in a very simple automata: every tick of its clock, update the world according to the rules we just described and notify the channel. One other thing this automata checks is the state of the diving turtles: since we only care about what the turtles do to the player, we only care about the moments in which they kill the player (submerged), and the moments they don't kill the player (submerging, emerging, emerged). This is handled by a boolean, `are_diving_turtles_up`, which gets set according to the time units described in the specification.

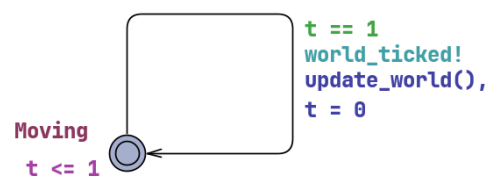
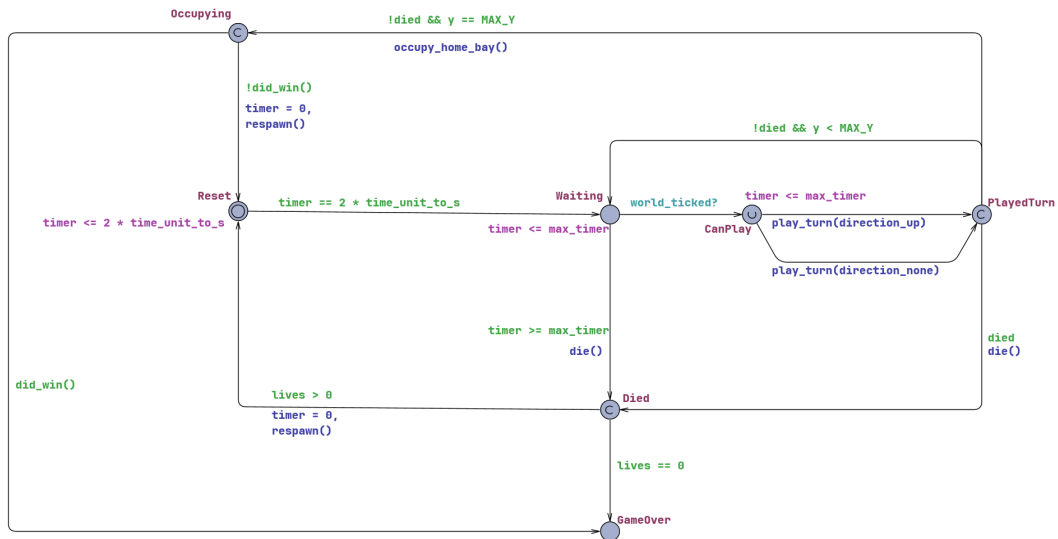


Figure 1: World automata

1.3. Player modeling (symbolic)

The player needs to handle a lot of different scenarios, so it will naturally be more complex:



We start in the **Reset** state, and we stay there for 2 seconds (which corresponds to 20 time units). After that time period, it goes in the **Waiting** state, from which it can exit only if one of two conditions is met:

- the `world_ticked` channel pushed a notification: we must perform the player turn
- the timer reached the maximum value allowed: it counts as player death

In the former, the turn is the one described in [Section 1.1](#). If the player reached the last row, it means it occupied a home bay: this means we must mark the home bay as occupied (hence making it unreachable in the next round), and check if we won the whole level. If that is the case, in the symbolic mode we must stop. The same applies if the player dies and its lives reach 0.

2. Symbolic version results

Note that there are two different system declarations: the “complete” one, and the “partial” one. For the former, ensure that in **System declarations** the line `system frogger, world;` is uncommented and `system world;` is commented; while for the latter the opposite must be verified. The first two queries will use “partial” system, while the others the “complete” one.

2.1. First query

ASSIGNMENT: the game never reaches a deadlock state.

QUERY USED:

A[] not deadlock

RESULTS: the query is trivially verified.

2.2. Second query

2.2.1. Assignment

ASSIGNMENT: all vehicles and floating objects can only move inside their predefined row.

QUERY USED:

```
A[] (offsets[1] >= 0 && offsets[1] < MATRIX_COLS &&
    offsets[2] >= 0 && offsets[2] < MATRIX_COLS &&
    offsets[3] >= 0 && offsets[3] < MATRIX_COLS &&
    offsets[4] >= 0 && offsets[4] < MATRIX_COLS &&
    offsets[5] >= 0 && offsets[5] < MATRIX_COLS &&
    offsets[7] >= 0 && offsets[7] < MATRIX_COLS &&
    offsets[8] >= 0 && offsets[8] < MATRIX_COLS &&
    offsets[9] >= 0 && offsets[9] < MATRIX_COLS &&
    offsets[10] >= 0 && offsets[10] < MATRIX_COLS &&
    offsets[11] >= 0 && offsets[11] < MATRIX_COLS)
```

RESULTS: The query is true by construction since the rows themselves move, but we can easily run it and see it is verified.

2.3. Third query

ASSIGNMENT: Frogger can reach the home bay in position (0,12)–(1,12).

QUERY USED:

```
E<> frogger.y == 12 && (frogger.x == 0 || frogger.x == 1)
```

RESULTS: easily verified: 0.324s

2.4. Fourth query

ASSIGNMENT: Frogger can score 17 points reaching the home bay in position (8,12)–(9,12).

QUERY USED:

```
E<> frogger.points == 17 && frogger.home_bays_occupied[2] &&
    frogger.y == 12 && (frogger.x == 8 || frogger.x == 9)
```

RESULTS: We need to check the point immediately after occupying the home bay, so we must check that the home bay is marked as occupied (so its points have been added), and the position is still the one required (no other movement). Verified in 0.245 s

2.5. Fifth query

ASSIGNMENT: Frogger can reach a yellow cell in row 6 after the player has lost two lives.

QUERY USED:

```
E<> frogger.lives == 3 && frogger.y == 6
```

RESULTS: Easily verified: 0.03s

2.6. Sixth query

ASSIGNMENT: Frogger may stand on a log (with five lives) 10 seconds after the game starts.

QUERY USED:

```
E<> frogger.lives == 5 &&  
    frogger.on_cell == C_LOG &&  
    global_clock == 10*time_unit_to_s
```

RESULTS: Verified: 3.756s

2.7. Seventh query

ASSIGNMENT: Frogger can die riding a diving turtle that transitions to the submerged state.

QUERY USED:

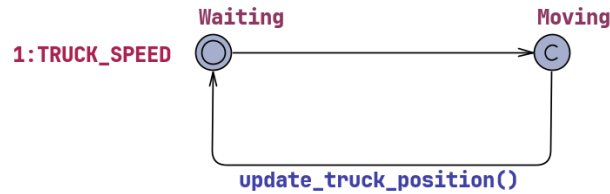
```
E<> frogger.died_by_submerging
```

RESULTS: This query is simple thanks to the `died_by_submerging` attribute: it is always false, except when we die while on a diving turtle before moving: this means the player died specifically because of the submerging of the turtles. Because of this, it is easily verified: 0.325s

3. Stochastic version

The modeling of the stochastic version is based on the symbolic one, so all the design choices described in [Section 1](#) remain valid, with some slight modifications. To handle the trucks moving randomly, we must

- Add a new array, representing the positions of the two trucks
- Implement a new automata:



This makes the automata exit the **Waiting** state with an exponential probability distribution having

$$\lambda = \frac{1}{\text{TRUCK SPEED}}$$

Every time a movement is triggered, the **update_truck_position** function is called: if the movement would not cause any overlap, it moves the truck to the next cell. We create two of these automata (as their movements must be unrelated to one another)

Finally, the **get_cell** function will perform an extra check: if the given row is the row of the trucks, instead of using the matrix and offsets array, we just use the offsets updated by the two **Truck** automata.

As for the **World** automata, there are no differences from the symbolic version, since the trucks are handled by separate automata.

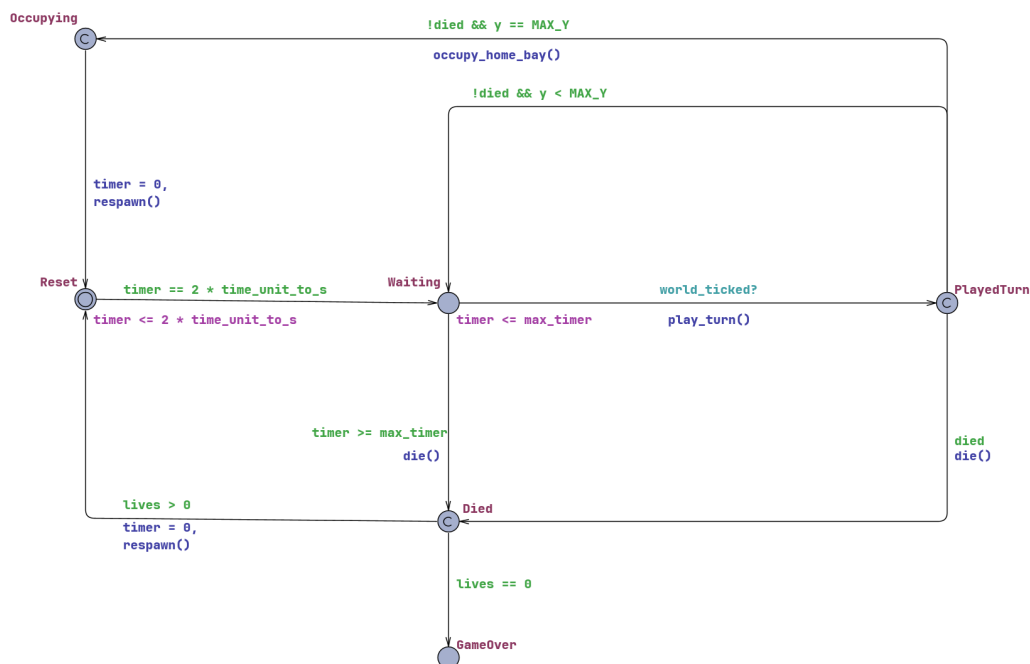
The player instead has some differences:

- Since here there can be more than one level, if the player wins a level it goes back to the **Reset** state, not to **GameOver**
- The movement is now done through the **play_turn** function: we decided to use a single transition in which a direction is chosen at random based on the assigned weights; as this allowed us to modify weights (as per [Uppaal specification](#), the weights must be constants, and that would not allow us to dynamically set the weights for the second query). Moreover, the specification also states that “an edge is still possible even if its weight happens to be zero” in symbolic simulation, which we found useful to debug unexpected behaviors of our system. To decide the movement’s direction, we defined the **choose_direction** function: it sums all the weights, and generates a random number from 0 to the sum of all weights using the **random** function (“pseudo random number distributed uniformly over the range [0, max).”). Then, it iterates over all the weights:

- ▶ if the current weights is greater than zero and greater than the random number, it chooses the movement associated with the weight
- ▶ otherwise, it subtracts to the generated number the weight and goes to the next one For example, let us assume that we have the following weights: `[0, 5, 1, 2, 2]` (direction none, up, down, left, right), and the generated number is 8. The function does the following checks:
 1. direction none (no movement): weight is not greater than zero, skip
 2. direction up: is 5 greater than 8? No, so the random number becomes $8 - 5 = 3$
 3. direction down: is 1 greater than 3? No, so the random number becomes $3 - 1 = 2$
 4. direction left: is 2 greater than 2? Yes, so the chosen direction is left

Note that this respects the weights, as numbers `{0, 1, 2, 3, 4}` means the choice is up, number `{5}` means choice is down, numbers `{6, 7}` mean choice is left, and number `{8, 9}` mean choice is right (number 10 is excluded from range).

Here is reported the new automata:



3.1. Queries

For the first query, the static policy must be used: to ensure that, under “System declarations”, ensure the line `frogger = Frogger(false);` is uncommented and the line `frogger = Frogger(true);` is commented.

3.1.1. First query

The assignment is “Given the above specification for the Stochastic version, simulate the game for a maximum of 1000 time units and estimate the probability of completing the first level while scoring at least 200 points”. To check the required property, we used the following query:

```
Pr[<=1000;5000] (<> (frogger.level == 2 && frogger.did_win() &&
frogger.points >= 200))
```

This ensures that:

- for a maximum of 1000 time units ($\text{Pr}[\leq 1000]$)
- repeat 1000 times ($; 5000$)
- check if a state exists such that:
 - the current level is 2
 - the player just won
 - the player has at least 200 points

This matches the assignment, as in our modeling, the function `occupy_home_bay()` will increase the level: this means that when the player wins the first level, the configuration will be with `level = 2`, and `did_win()` returns true.

The results of the query is that in 0 runs out of 5000 the requirements are verified. If we use the dynamic policy, we get a bit more successes (31): this is due to the points requirement. Since our player will always prefer moving up if it is safe, and the points are 1 per movement up, 5 per home bay, and 100 per level won, if we never go back down we get:

- 12 times 1 point for going up
- 5 times 5 points for the home bays
- 1 time 100 points for winning the level

For a total of 136 points. As a matter of fact, if we run the query without the points requirement, with the static policy we still get 0 successes, while with the dynamic one we get 4965 successes. In general, our dynamic policy strongly prefers survival and winning over point scoring, so any strict requirement on points will bring the number of successes down significantly.

3.1.2. Second query

To better illustrate the improvements our dynamic policy presents over the static one, we used the following queries:

```
simulate[<=600; 10] {frogger.points}
simulate[<=600; 10] {frogger.level}
```

```
simulate[<=600; 10] {frogger.lives}
```

We then ran them for both the static and dynamic policy, and compared the obtained results (each simulation is of a different color only for better readability):

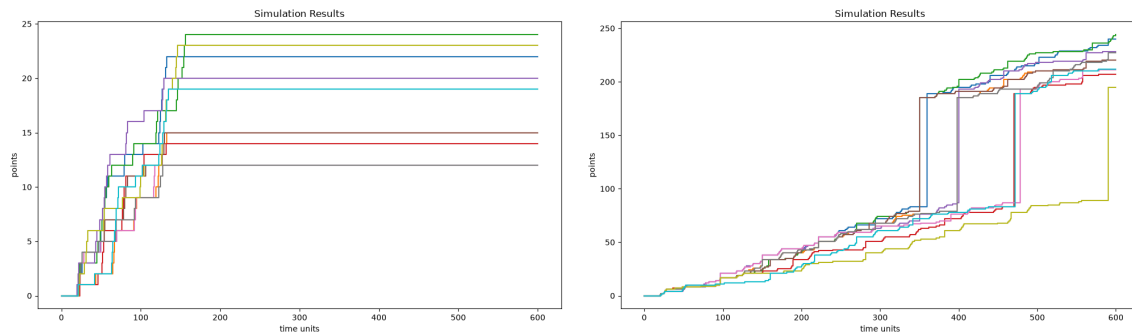


Figure 5: Comparison of the static and dynamic policy for the points scoring

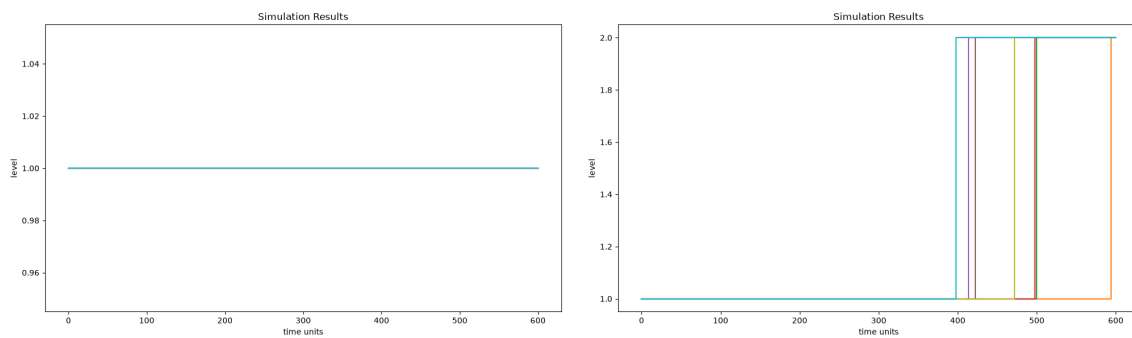


Figure 6: Comparison of the static and dynamic policy for the level reached

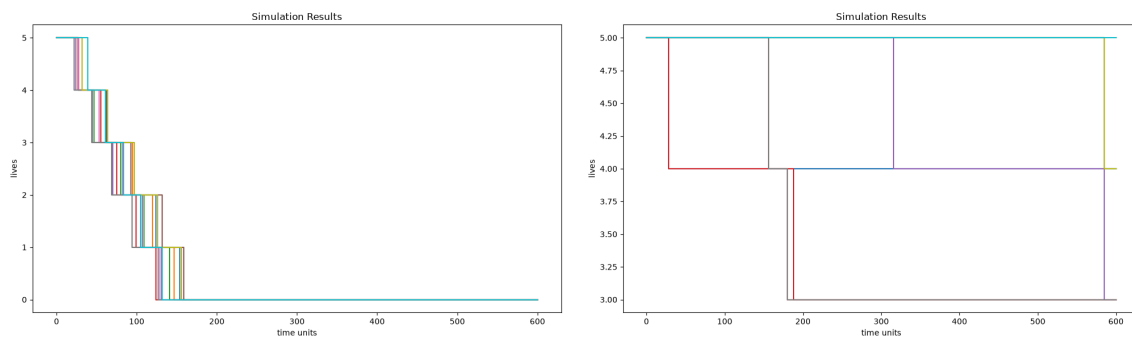


Figure 7: Comparison of the static and dynamic policy for the remaining lives

As we can see, the dynamic policy reaches higher levels (if we increase the time interval to 1000, we almost always get to level 3 and often 4), with more lives. Points are also higher in the long run, because completing levels gives more points than going up and down: we can see we have a lot of steep increases, that correspond to

the level completion (which we do not observe in the static version, as we can see from the scale of the two graphs).

It is also worth noting that this holds true for the lower levels: since our policy prefers vertical movement, it often happens that in the safe line separating the “grey” lines from the “light blue” ones, the player has to wait for a long time for the floating object to be in front of him, making him lose frequently at higher levels where the timer is much shorter. This could be solved with a more complex policy that, when the player is on that row:

- checks the relative position of the floating object with respect to the player
- knowing the direction of movement of the next row, updates the weights so that the player goes in the direction that guarantees the lowest amount of time before the player can jump on the floating object (the row is safe, so horizontal movement is always safe)

Bibliography

- [1] K. G. Larsen, P. Pettersson, and W. Yi, “UPPAAL documentation,” [Online]. Available: <https://docs.uppaal.org/>
- [2] K. G. Larsen, P. Pettersson, and W. Yi, “UPPAAL,” [Online]. Available: <https://uppaal.org/>

We also found useful the following resources:

- [This youtube tutorial](#) to understand the building blocks of the UPPAAL tool and their functioning
- [This atari emulator](#) to play the game and better understand its mechanics